

Program No. 10

Client-Server Communication Using UDP

Aim

To implement UDP-based client-server communication in which the client inputs a sentence written in informal, new-generation English (containing abbreviations such as tbh, ig, idk, and so on) and sends it to the server. The server scans the sentence, replaces every recognized abbreviation with its formal English expansion, and sends the translated sentence back to the client, which then displays it.

Algorithm – Client

1. Start the client program and import the socket module, which provides the functions needed for network communication.
2. Create a UDP socket using `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`, where `AF_INET` selects IPv4 addressing and `SOCK_DGRAM` specifies connectionless, datagram-based communication.
3. Store the server's IP address (127.0.0.1) and port number (9002) together as a tuple, to be used as the destination address for the datagram.
4. Prompt the user to enter a new-generation English sentence containing informal abbreviations, using `input()`.
5. Send the entered sentence to the server using `sendto()`, passing the encoded sentence and the server's address tuple.
6. Wait to receive the server's reply using `recvfrom()`, which returns the translated sentence along with the server's address.
7. Decode and display the received, expanded sentence on the client's console.
8. Close the socket to release the resources used for communication.

Algorithm – Server

1. Start the server program and import the socket module.
2. Define a dictionary that maps each known abbreviation (tbh, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm, and idc) to its formal English expansion, so the translation logic is expressed as a simple lookup instead of manual string searching.

3. Create a UDP socket using `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`.
4. Bind the socket to address "0.0.0.0" (all available network interfaces) and port 9002, so the server can be reached by the client.
5. Print a message indicating that the server is running and waiting to receive a message, so the server's progress is visible.
6. Wait to receive a datagram from the client using `recvfrom()`, which also returns the client's address so a reply can be sent back; decode and print the received sentence to confirm it arrived correctly.
7. Iterate over the dictionary of abbreviations and use a word-boundary regular expression (`re.sub()` with `\b` markers) to substitute every matching abbreviation in the message with its formal expansion, ensuring that only whole-word matches are replaced.
8. Print a message indicating that the translated sentence is about to be sent back to the client.
9. Encode the translated sentence and send it back to the client using `sendto()`, addressed to the client using the address captured earlier by `recvfrom()`.
10. Close the socket to release the resources used for communication.

Client Program

```
import socket

# Create a UDP socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

server_address = ("127.0.0.1", 9002)

sentence = input("Enter sentence:\n")

# Send the sentence to the server
client.sendto(sentence.encode(),
server_address)

# Wait for the server's reply
data, _ = client.recvfrom(2048)

print("\nExpanded
form:")
print(data.decode())

client.close()
```

Server Program

```
import
socket
import re

# Dictionary of abbreviations and their formal expansions
abbreviations = {
    "tbh": "to be honest",
    "ig": "I guess",
    "tbf": "to be fair",
    "atm": "at the
moment", "irl": "in
real life",
    "lol": "laughing out loud",
    "asap": "as soon as
possible", "omg": "oh my
God",
    "ttyl": "talk to you later",
    "idk": "I don't know",
    "nvm": "never mind",
    "idc": "I don't care",
}

# Create a UDP socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("0.0.0.0", 9002))

print("UDP Server is running and waiting for a

message...") # Wait to receive a datagram from the client
data, client_address = server.recvfrom(2048)
message = data.decode()
print("Received from client:", message)

# Replace every known abbreviation with its formal
expansion # (\b...\b matches whole words only, so
abbreviations are not
# wrongly matched inside longer words, e.g. "ig" inside
"design") for short_form, expansion in abbreviations.items():
    pattern = r"\b" + re.escape(short_form) +
    r"\b" message = re.sub(pattern, expansion,
    message)

print("Sending translated message back to client...")
server.sendto(message.encode(), client_address)

server.close()
```

Result

Client Side

Enter sentence:

Really idc about this stupid server as it is of no use irl but atm, I will design one, tbf to the professor.

Expanded form:

Really, I don't care about this stupid server as it is of no use in real life but at the moment, I will design one, to be fair to the professor.

Server Side

UDP Server is running and waiting for a message...

Received from client: Really idc about this stupid server as it is of no use irl but atm, I will design one, tbf to the professor.

Sending translated message back to client...